

Mastering Actor-Critic Architectures & GAE

Complete Mathematical & Algorithmic Foundations: Policy Evaluation, Temporal Difference Bootstrapping, Generalized Advantage Estimation Telescoping Derivation, Squashed Continuous Control, Asymmetric Actor-Critic, and PyTorch Vectorized Implementations

Instructor: Prof. Sergey Levine **Curriculum:** Berkeley CS285 + Schulman et al. (GAE) **Scope:** General Continuous Actor-Critic (UC Berkeley) + Achiam (Spinning Up) Theory & Robotics Slicing

0. The Executive Mental Map: Why Does Lecture 6 Exist?

In Lecture 5, we saw that subtracting a state-dependent baseline $b(s_t) = V(s_t)$ reduces policy gradient variance without introducing bias. However, in standard REINFORCE with a baseline, we still compute advantage using **full Monte Carlo rollouts** to the end of the episode:

$$\hat{A}_t = \sum_{t'=t}^T \gamma^{t'-t} r_{t'} - V_{\phi}(s_t)$$

The Critical Vulnerability: In long-horizon continuous robotics tasks (such as delicate surgical dissection or contact manipulation), relying on full episode rollouts means a single unpredictable disturbance at step $t = 195$ corrupts the learning signal for all preceding 194 steps.

Lecture 6 introduces the Actor-Critic paradigm: Instead of waiting for the future to happen, the agent trains a dedicated **Critic neural network** to predict the future via **bootstrapping and Temporal Difference (TD) learning**. The **Actor** can then update policy parameters at every single time-step, eliminating Monte Carlo variance.

1. Actor-Critic

Actor (Student) +
Critic (Coach)

2. TD Bootstrapping

1-Step Error δ_t
 $r + \gamma V(s') - V(s)$

3. Bias-Variance

Monte Carlo (Noise) vs
1-Step TD (Bias)

4. GAE- λ Telescoping

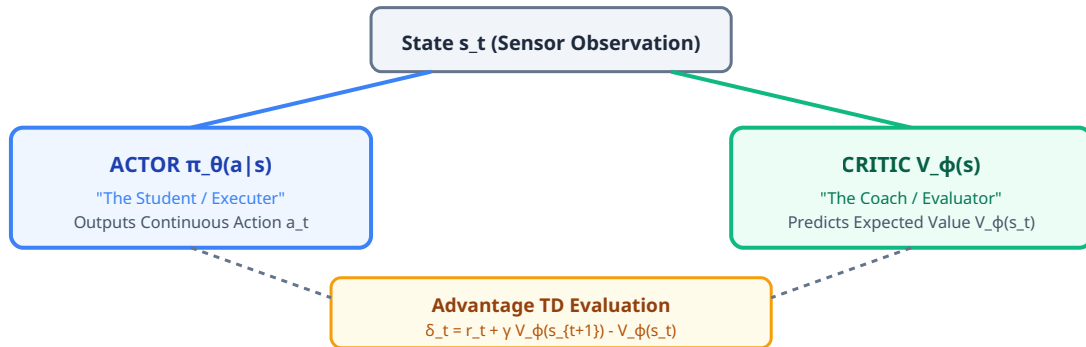
Exponential Horizon
Optimal $\lambda = 0.95$

5. Asymmetric AC

Privileged Critic Training
Deployable Real Actor

Part 1: The Core Architecture — Actor (Student) & Critic (Coach)

(Slides 1–15) An Actor-Critic algorithm decouples the decision-making policy from the value prediction network into two specialized systems operating in closed-loop synergy:



1.1 Mathematical Definitions of the Value Triad

Function	Mathematical Formula	Physical Role in Robotic Manipulation
State Value $V^\pi(s)$	$\mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s \right]$	Evaluates state promise: "Blade is aligned with zero shear strain; expected cumulative return is +65."
Action Value $Q^\pi(s, a)$	$\mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s, a_0 = a \right]$	Evaluates action consequence: "Taking downward feed velocity $v_z = 2 \text{ mm/s}$ right now yields +80."
Advantage Function $A^\pi(s, a)$	$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$	Relative superiority of action: $A = 80 - 65 = +15$. The action was substantially better than average!

Part 2: Value Function Fitting & Temporal Difference Bootstrapping

(Slides 16–35) How does the Critic learn to predict $V(s)$ accurately?

2.1 Monte Carlo vs. Temporal Difference (TD) Targets

1. **Monte Carlo Target:** Wait for full episode completion to observe true total return $y_t^{\text{MC}} = \sum_{t'=t}^T \gamma^{t'-t} r_{t'}$.

$$\mathcal{L}_{\text{Critic}}(\phi) = \frac{1}{2} \mathbb{E} [(V_{\phi}(s_t) - y_t^{\text{MC}})^2]$$

Tradeoff: Strictly unbiased, but massive variance because y_t^{MC} sums hundreds of random future state transitions.

2. **1-Step Temporal Difference (TD) Bootstrapping Target:** Step forward just **1 single time-step**, observe immediate reward r_t , and bootstrap from the Critic's own prediction at the next state s_{t+1} :

$$y_t^{\text{TD}} = r_t + \gamma V_{\phi}(s_{t+1})$$

$$\mathcal{L}_{\text{Critic}}(\phi) = \frac{1}{2} \mathbb{E} [(V_{\phi}(s_t) - [r_t + \gamma V_{\phi}(s_{t+1})])^2]$$

Tradeoff: Near-zero variance, but biased early in training when V_{ϕ} is poorly fitted.

2.2 The 1-Step TD Error (δ_t^V) as an Advantage Estimator

$$\delta_t^V = r_t + \gamma V_{\phi}(s_{t+1}) - V_{\phi}(s_t)$$

Notice that δ_t^V is an unbiased 1-step sample estimator of the true Advantage function:

$$\mathbb{E}[\delta_t^V \mid s_t, a_t] = \mathbb{E}[r_t + \gamma V(s_{t+1}) \mid s_t, a_t] - V(s_t) = Q(s_t, a_t) - V(s_t) = A(s_t, a_t)$$

THE SEMI-GRADIENT DETACH TRAP

In TD learning, the target $y = r + \gamma V_{\phi}(s_{t+1})$ is treated as a **fixed regression target** derived from the Bellman operator. In PyTorch, you must write:

```
target = (reward + gamma * critic(next_state).detach())
```

If you omit `.detach()`, PyTorch computes gradients through both $V(s_t)$ and $V(s_{t+1})$, transforming TD regression into a broken optimization problem where the network chases its own tail, resulting in eigenvalue explosion and divergence.

Part 3: The Bias-Variance Dilemma & N-Step Returns

(Slides 36–48) Neither pure Monte Carlo nor 1-step TD is fully satisfactory across all robotics tasks:

- **Monte Carlo (∞ -step):** Zero bias, but extreme variance. Soft tissue contact noise compounds endlessly across hundreds of steps.
- **1-Step TD:** Minimal variance, but biased if the Critic's prediction is imperfect early in training.
- **N-Step Return:** Looks n steps ahead before bootstrapping:

$$G_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n V(s_{t+n})$$

Part 4: Generalized Advantage Estimation (GAE- λ)

(Slides 49–65) In their landmark paper *High-Dimensional Continuous Control Using Generalized Advantage Estimation*, John Schulman, Sergey Levine et al. (2016) proposed a mathematically unified method that smoothly interpolates between 1-step TD and full Monte Carlo.

4.1 Mathematical Derivation of the Telescoping GAE Sum

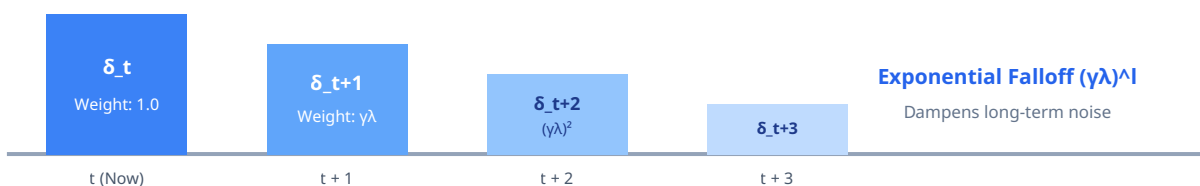
Define the k -step advantage estimator $\hat{A}_t^{(k)}$:

$$\begin{aligned}\hat{A}_t^{(1)} &= \delta_t^V = r_t + \gamma V(s_{t+1}) - V(s_t) \\ \hat{A}_t^{(2)} &= \delta_t^V + \gamma \delta_{t+1}^V = r_t + \gamma r_{t+1} + \gamma^2 V(s_{t+2}) - V(s_t) \\ \hat{A}_t^{(k)} &= \sum_{l=0}^{k-1} \gamma^l \delta_{t+l}^V = \sum_{l=0}^{k-1} \gamma^l r_{t+l} + \gamma^k V(s_{t+k}) - V(s_t)\end{aligned}$$

Notice how all intermediate value predictions telescope and cancel out identically! Schulman et al. define the ****Generalized Advantage Estimator**** as the exponentially weighted average of all k -step estimators, parameterized by $\lambda \in [0, 1]$:

The GAE Telescoping Identity:

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda)} = (1 - \lambda) \sum_{k=1}^{\infty} \lambda^{k-1} \hat{A}_t^{(k)} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}^V$$



4.2 Geometric Decay and the λ Tuning Spectrum

- $\lambda = 0$: $\hat{A}_t^{\text{GAE}} = \delta_t^V$. Strictly 1-step TD. Lowest variance, highest bias.
- $\lambda = 1$: $\hat{A}_t^{\text{GAE}} = \sum_{l=0}^{\infty} \gamma^l \delta_{t+l}^V = \sum_{l=0}^{\infty} \gamma^l r_{t+l} - V(s_t)$. Full Monte Carlo return minus baseline. Zero bias, highest variance.
- **The Universal Gold Standard:** $\lambda = 0.95, \gamma = 0.99$. Setting $\lambda = 0.95$ provides the optimal bias-variance Pareto frontier for robotics, allowing credit assignment over roughly $1/(1 - \lambda) \approx 20$ to 40 steps while dampening distant contact noise.

Part 5: Continuous Control & Squashed Gaussian Policies

(Slides 66–78 & Spinning Up ch19) In continuous robotics, actions are physically bounded (e.g., motor velocities cannot exceed physical limits). How do we bound continuous Gaussian policies?

5.1 The Tanh Squashing Transformation & Jacobian Correction

Instead of clipping actions (which destroys gradients outside the boundary), we apply a smooth hyperbolic tangent squashing function: $a = \tanh(u)$, where $u \sim \mathcal{N}(\mu_\theta(s), \sigma_\theta(s))$. Because $\tanh$ is a non-linear change of variables, the probability density transforms according to the Jacobian determinant:

$$P(a \mid s) = P(u \mid s) \cdot \left| \det \left(\frac{da}{du} \right) \right|^{-1}$$

Taking the logarithm:

$$\log \pi(a \mid s) = \log \mu(u \mid s) - \sum_{i=1}^d \log (1 - \tanh^2(u_i) + \epsilon)$$

Part 6: Asymmetric Actor-Critic for Modern Robotics

THE ASYMMETRIC INFORMATION PRINCIPLE (ISAAC GYM / ISAAC LAB)

During deployment on a physical robot, the Actor policy must act solely on realistic, noisy sensor inputs: joint encoders, tactile pressure arrays, and camera RGB-D images ($o_t \in \Omega$).

However, **during simulation training, the Critic is never deployed on the physical robot!** The Critic is purely an offline coach used to evaluate states. Therefore, we can feed the Critic **privileged ground-truth simulator states** s_t : exact tissue friction coefficients, internal stress tensor fields, cutting board reaction torques, and true blade penetration depth!

The Benefit: The Critic learns a near-perfect value landscape without sensory ambiguity, providing ultra-clean advantage signals that guide the sensory-restricted Actor to master delicate cutting!

Part 7: Formal Algorithm Specification & PyTorch Implementation

ALGORITHM 2: SYNCHRONOUS ADVANTAGE ACTOR-CRITIC (A2C) WITH GAE

Initialize: Policy parameters θ , Critic parameters ϕ , hyperparams $\gamma = 0.99, \lambda = 0.95, c_1 = 0.5, c_2 = 0.01$.

1. **for** iteration $k = 1, 2, \dots$ **do**
2. Collect T transitions across M parallel GPU environments using current policy π_θ .
3. Evaluate Critic values $V_\phi(s_t)$ and terminal bootstrap value $V_\phi(s_{T+1})$.
4. Compute 1-step TD errors: $\delta_t^V = r_t + \gamma(1 - d_t)V_\phi(s_{t+1}) - V_\phi(s_t)$.
5. Compute GAE advantages recursively backwards from $t = T$ down to 1:

$$\hat{A}_t = \delta_t^V + \gamma\lambda(1 - d_t)\hat{A}_{t+1}$$

6. Compute value targets: $\hat{R}_t = \hat{A}_t + V_\phi(s_t)$.
7. Normalize advantages across batch: $\hat{A} \leftarrow \frac{\hat{A} - \text{mean}(\hat{A})}{\text{std}(\hat{A}) + 10^{-8}}$.
8. Compute composite loss:

$$\mathcal{L}(\theta, \phi) = -\frac{1}{M \cdot T} \sum \left[\log \pi_\theta(a_t | s_t) \hat{A}_t - c_1 (V_\phi(s_t) - \hat{R}_t)^2 + c_2 \mathcal{H}(\pi_\theta(\cdot | s_t)) \right]$$

9. Update (θ, ϕ) via Adam optimizer.
10. **end for**

7.1 Production Vectorized PyTorch GAE Computation

```
import torch

def compute_gae_vectorized(rewards, values, next_values, dones, gamma=0.99, lam=0.95):
    # rewards, values, next_values, dones: Shape [T, N]
    T = rewards.size(0)
    advantages = torch.zeros_like(rewards)
    last_gae = 0.0

    for t in reversed(range(T)):
        non_terminal = 1.0 - dones[t]
        delta = rewards[t] + gamma * next_values[t] * non_terminal - values[t]
        advantages[t] = last_gae = delta + gamma * lam * non_terminal * last_gae

    returns = advantages + values
    # Normalize advantage across batch
    norm_adv = (advantages - advantages.mean()) / (advantages.std() + 1e-8)
    return norm_adv, returns
```

Part 8: Interactive Tablet Self-Test Quiz

Question 1: What happens to GAE when you set $\lambda = 0$, and what happens when you set $\lambda = 1$?

Answer: When $\lambda = 0$, GAE reduces strictly to 1-step TD error ($\delta_t = r_t + \gamma V(s') - V(s)$), giving lowest variance but highest bias. When $\lambda = 1$, GAE telescopes into the full empirical Monte Carlo return minus baseline ($\sum \gamma^l r_{t+l} - V(s)$), giving zero bias but maximum variance.

Question 2: Why must the Critic network be trained with semi-gradients (detaching the target)?

Answer: The Bellman target $y = r + \gamma V(s')$ is derived from dynamic programming, where $V(s')$ serves as a fixed reference value. If we backpropagate gradients into $V(s')$, the network will try to move both its current prediction and future target simultaneously, causing eigenvalues to drift and resulting in mathematical instability.

Question 3: Why does setting $\lambda = 0.95$ prevent the robot from misinterpreting a skin puncture reward?

Answer: Skin puncture is an abrupt transition that yields a high localized reward. Setting $\lambda = 0.95$ focuses the advantage assignment on the 20–40 steps directly surrounding the rupture event, rewarding the preparatory deceleration and sawing actions without smearing noise across the subsequent 160 steps of pulp slicing.

Part 9: Thesis Defense Master Cheatsheet (Lecture 6 Focus)

Q1: "WHAT IS THE PRIMARY ADVANTAGE OF ACTOR-CRITIC OVER PURE POLICY GRADIENTS (REINFORCE)?"

Answer: "REINFORCE relies on empirical Monte Carlo returns, requiring complete trajectory rollouts. This introduces immense variance, as a single disturbance late in the episode pollutes credit assignment for all earlier steps. Actor-Critic introduces a learned Critic $V_\phi(s)$ that evaluates states via 1-step Temporal Difference bootstrapping ($r + \gamma V(s')$), drastically reducing variance and enabling stable, sample-efficient learning in contact-rich tasks."

Q2: "EXPLAIN THE BIAS-VARIANCE TRADEOFF BETWEEN MONTE CARLO EVALUATION AND 1-STEP TD LEARNING."

Answer: "Monte Carlo evaluation has zero bias because it measures true realized rewards, but suffers from extreme variance due to compounding stochastic transitions. 1-step TD learning has minimal variance because it only looks one step ahead, but introduces bias because it relies on the Critic's own function approximation $\hat{V}(s_{t+1})$, which may be inaccurate early in training."

Q3: "WHY IS GAE- λ UNIVERSALLY PREFERRED FOR CONTINUOUS ROBOTIC MANIPULATION LIKE TOMATO SLICING?"

Answer: "Robotic cutting involves abrupt physical regime shifts (e.g., non-contact approach $\rightarrow$ elastic skin indentation $\rightarrow$ rupture $\rightarrow$ flesh slicing). Pure 1-step TD is too myopic to anticipate delayed fruit deformation, while pure Monte Carlo is overwhelmed by contact noise. GAE- λ exponentially weights multi-step returns with parameter $\lambda = 0.95$, providing the optimal balance: it reacts quickly to localized rupture events while maintaining a sufficient temporal horizon to optimize smooth compliant trajectories."